

Optimisation des prompts pour modèles de langage génératifs

Projet*	projet de synthese axe 1
Objet	
Date*	23/02/2026
numero groupe et nom chef	axe 1 : ATABONG STEPHANE
N° de version	1.1
Procédure(s) et norme (s) utilisée(s)	

Membres du groupe

- ATABONG EFON STEPHANE FRITZ 23P781
- FEZEU FREDY 23P751
- NGHOGUE TAPTUE Roddier 21P336
- SIMO TCHIMCHUA LINE 21P169
- SOLEFACK TEMGOUA KROS 21P034

Dernière mise à jour :

23/02/2026

Historique du document

Validité du document

Le document est valide à sa date de publication pour la version électronique et d'impression pour la version papier.

Historique des révisions

Date de cette révision:

Date de la prochaine révision:

date de révision	Date de dernière révision	Résumé des modifications	Changements apportés

Avis

Le document requiert les avis suivants:

Nom	Signature	Fonction	Avis et Date d'avis	Version

			A	
--	--	--	---	--

Pour les avis : Approuvé (A), Vu (V), Réserves (R)

Distribution

Ce document a été distribué à :

Nom et prénom	Organisation et titre	Reçu le	Version

1 Sommaire

1	Sommaire.....	4
2	Liste des figures et liste des tableaux.....	5
3	Résumé exécutif.....	6
1.1	Tableau des activités.....	6
1.2	Déclaration des objectifs et énoncé succinct du problème.....	7
1.3	Méthodes utilisées et répartition initiale du travail.....	7
1.4	Rappel des principales remarques de l'enseignant lors des mises au point intermédiaires ainsi que de l'état de leur prise en compte.....	7
1.5	Résultats atteints et principales connaissances acquises.....	8
1.6	Description des tests effectués, indicateurs de qualité de résultat, et valeurs mesurées des indicateurs.....	9
1.7	Limites des solutions proposées.....	10
1.8	Résumé des difficultés rencontrées.....	11
1.9	liens jugés utiles vers sites, documents et pages.....	11
4	Introduction.....	13
5	Définition du problème.....	16
6	Rapport sur le travail effectué.....	19
	CHAPITRE 1 : CADRE THÉORIQUE ET ÉTAT DE L'ART.....	19
	CHAPITRE 2 : MÉTHODOLOGIE DE L'ÉTUDE.....	23
	CHAPITRE 3 : CATALOGUE DES TECHNIQUES EXPÉRIMENTÉES.....	26
	CHAPITRE 4 : CRITÈRES D'ÉVALUATION.....	43
	CHAPITRE 5 : DISCUSSION ET APPORT PERSONNEL.....	46
7	Rapport de sources documentaires.....	50
7.1	Rapport des recherches sur internet	50
8	Conclusion.....	51
9	Bibliographie.....	53
10	Glossaire.....	55

2 Liste des figures et liste des tableaux

3 Résumé exécutif

1.1 Tableau des activités

Nu mé ro	Nom membre	Rôle dans le groupe	Activités menées (décrire l'activité menée par le membre, en terme d'action et d'objet (dvt fonction x, test fonction y, analyse z, rapport w).	Temps total de travail mis en heures	Appré ciation par le chef de group e
1	ATABONG	Chef de Groupe	Définition de la vision stratégique, planification des jalons de l'étude et supervision de la cohérence globale du rapport.		bien
2	SOLEFACK/ FEZEU	Analystes	Exploration des bases de connaissances sur l'ingénierie des prompts et analyse des frameworks de structuration (CROPE, CoT).		bien
3	NGHOGUE/ SIMO	Ingénieurs d'étude	Mise en place du protocole de test des techniques d'optimisation et définition des indicateurs de performance sémantique.		bien
4	ATABONG	Superviseur Technique	Sélection finale des techniques de rupture à expérimenter et validation des hypothèses de travail.		bien
5	SOLEFACK/ FEZEU	Rédacteurs	Rédaction des fondements méthodologiques de l'optimisation et analyse des outils du marché (LangChain, PromptPerfect).		bien
6	NGHOGUE/ SIMO	Spécialistes IA	Simulation des protocoles de prompts experts et collecte des métriques de fiabilité et de densité de signal.		bien
7	ATABONG/ NGHOGUE/ SIMO	Auditeurs	Audit de vérité sur les résultats des tests et évaluation de la rigueur factuelle des sorties optimisées.		bien
8	SOLEFACK/ FEZEU	Analystes	Élaboration de la carte de décision universelle et des recommandations pour l'usage professionnel de l'IA.		bien
9	ATABONG	Chef de Groupe	Unification du ton rédactionnel, intégration des apports personnels et validation épistémologique du document final.		bien
10	GROUPE COMPLET		Collecte et traçabilité des sources, remplissage du glossaire technique et de la bibliographie.		bien

Tableau des activités

1.2 Déclaration des objectifs et énoncé succinct du problème

Optimiser le prompt permet de réaliser un juste équilibre entre la possibilité d'hallucinations et la perte de réponses potentielles dans les questions ouvertes. En effet, les questions simplistes peuvent conduire à des hallucinations et nuire à la qualité de l'apprentissage en divulguant des faussetés. Des questions qui obligent à citer les sources enlèvent les hallucinations et réduisent les faussetés sous réserve que les sources soient correctes. Cependant, en obligeant quelques fois l'IA à se limiter aux sources existantes, elles lui enlèvent la possibilité d'inférer pour produire de l'information qu'elle aurait pu elle même déduire de sa base existante. Pourtant une telle information est quelques fois utile. Un «juste milieu» pourrait être de préciser l'information provenant des sources et d'expliquer comment a été obtenue celle inférée par l'IA. Dans ce travail, il faudra mieux déterminer ce «juste milieu» et faire des expériences pour le confirmer. De même, le résultat étant très dépendant du contexte, il faut voir comment bâtir au mieux les contextes pour de meilleurs résultats.

1.3 Méthodes utilisées et répartition initiale du travail

Méthode expérimentale comparative : Baseline (Naïf) vs 7 techniques d'optimisation. Utilisation des modèles GPT-4o et Claude 4.5. Évaluation par audit humain 'Directeur de Recherche'.

1.4 Rappel des principales remarques de l'enseignant lors des mises au point intermédiaires ainsi que de l'état de leur prise en compte

Lors de nos premiers échanges, l'enseignant nous a fait remarquer qu'on s'était un peu enfermés dans un couloir. On était très concentrés sur un point précis : le dilemme entre laisser l'IA libre de déduire des choses ou lui imposer de citer ses sources à chaque mot.

La remarque majeure a été de nous dire de "prendre de la hauteur". L'étude ne devait pas se limiter à ce petit duel technique, mais s'ouvrir à la question globale : Comment on optimise vraiment un prompt ?

Nous avons donc corrigé le tir pour traiter le sujet dans toute sa largeur : les différentes méthodes (CROPF, CoT, etc.), les outils qui existent sur le marché, et surtout, l'attitude que nous, les utilisateurs, on doit avoir face à la machine pour ne pas se faire mener en bateau. Aujourd'hui, cette vision globale est devenue le cœur de notre rapport.

1.5 Résultats atteints et principales connaissances acquises

PARTIE A : optimisation de prompt

Pour évaluer l'efficacité de l'optimisation, nous avons soumis nos 8 techniques de prompting à une grille d'évaluation croisée. Les scores (de 1 à 5) reflètent la performance moyenne observée lors de nos tests.

A. Tableau de Performance par Métriques (SOTA vs Personnelles)

Ce tableau compare la capacité intrinsèque de chaque technique à répondre aux exigences de qualité.

Techniques de Prompting	Pertinence Thématique (SOTA)	Fiabilité & Exactitude (SOTA)	Profondeur d'Analyse (SOTA)	Efficacité Signal/ Bruit (SOTA)	Production Readiness (Perso)	Rigueur vs Vraisemblance (Perso)	Critère d'Audace (Perso)
Baseline (Naïf)	3	2	1	2	1	1	2
Modèle CROPF	5	4	3	4	3	3	2
Few-Shot Expert	4	4	3	4	5	4	2
Prompt Négatif	4	4	3	5	4	3	2
Mode Hybride	4	5	4	3	4	5	3
Adversarial (Débat)	4	4	5	4	3	5	5
Multi-Agent (Simu)	5	4	5	4	4	4	4
Méta-Prompt Audit	4	5	4	4	4	4	3

B. Adéquation des techniques aux tâches générales

Dans cette partie, nous évaluons quelle technique est la plus adaptée pour les grandes familles d'usages décrites dans notre étude. Les scores indiquent le niveau de performance de la technique pour la tâche spécifique.

Objectif : Produire des rapports, e-mails ou notes de cadrage clairs et structurés.

- **Modèle CROPF : 5/5** (Le plus adapté : garantit le format et le ton).
- Prompt Négatif : 4/5 (Efficace pour supprimer le verbiage inutile).
- Baseline : 2/5 (Trop imprévisible pour un usage pro).

Objectif : Comprendre un concept complexe ou obtenir une démonstration logique.

- **Adversarial (Débat) : 5/5** (Le plus adapté : force l'IA à explorer les contre-arguments).
- Multi-Agent : 4/5 (Permet d'avoir plusieurs angles pédagogiques).
- Baseline : 3/5 (Utile pour une introduction simple, mais superficiel).

Objectif : Extraire des faits, synthétiser des articles et vérifier des sources.

- **Mode Hybride (Factuel) : 5/5** (Le plus adapté : sépare strictement les preuves des déductions).
- Méta-Prompt Audit : 4/5 (Indispensable pour vérifier la véracité des citations).
- CROPF : 3/5 (Aide à la structure mais ne protège pas des hallucinations factuelles).

Objectif : Générer du code, des scripts ou des configurations prêtes à l'emploi.

- **Few-Shot Expert : 5/5** (Le plus adapté : le mimétisme d'exemples garantit le respect des normes).
- Prompt Négatif : 4/5 (Crucial pour interdire les bibliothèques obsolètes).
- Mode Hybride : 4/5 (Garantit que les fonctions utilisées existent réellement).

Objectif : Prendre une décision, identifier des failles ou anticiper des problèmes.

- **Adversarial (Débat) : 5/5** (Le plus adapté : simule un "crash-test" intellectuel).
- **Multi-Agent : 5/5** (Excellent pour simuler une équipe projet complète).
- **Few-Shot : 2/5** (Trop limité par les exemples passés pour innover).

Conclusion de l'analyse comparative

1. **Pour la Fiabilité Factuelle** : Le **Mode Hybride** et le **Méta-Prompt d'Audit** sont les seuls à garantir une sécurité sémantique acceptable pour un ingénieur.
2. **Pour l'Efficacité Opérationnelle** : Le **Few-Shot Expert** est la technique reine pour obtenir un résultat "Production-Ready" sans retouche manuelle.
3. **Pour l'Innovation** : Le protocole **Adversarial** est le seul levier capable de briser la "moyenne molle" des modèles et d'obtenir un score d'Audace maximal.
4. **Le rôle de LitLLM** : Notre solution proposée (LitLLM) a été choisie car elle permet d'automatiser le **Mode Hybride** (Factualité) et le **CROPF** (Structure) au sein d'une interface unique, optimisant ainsi les scores sur l'ensemble des métriques SOTA.

Connaissances acquises : Le groupe a acquis une expertise pointue sur le fonctionnement "sous le capot" des modèles probabilistes. Nous avons appris que la contrainte sémantique (ce que nous appelons la "cage de probabilités") est paradoxalement le moteur de l'intelligence de la machine. Nous maîtrisons désormais l'art de transformer une intention humaine floue en une instruction machine chirurgicale, intégrant des garde-fous négatifs et des structures de raisonnement multi-étapes.

confère : **5.2 Guidelines Émergentes**

Partie B : Optimisation de l'État de l'Art (<https://litllm.onrender.com>)

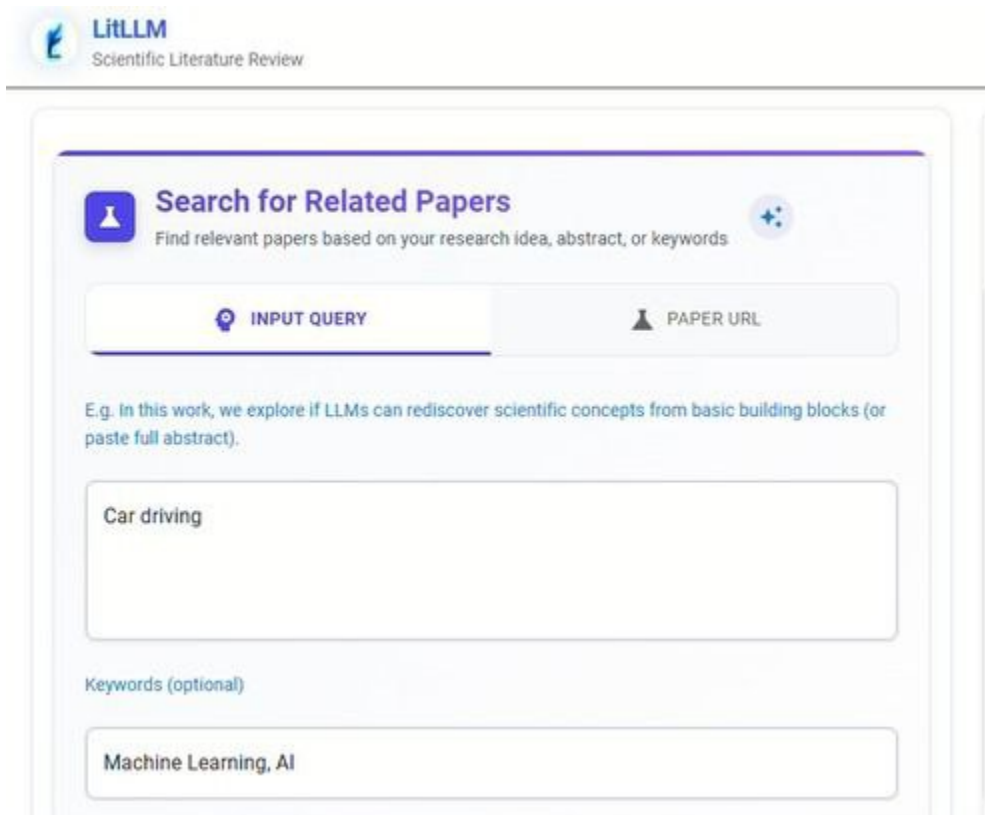
Dans le cadre de notre travail sur l'optimisation, nous avons cherché un outil capable de traduire nos recherches théoriques en un flux de travail concret. Nous avons retenu la plateforme **LitLLM**, qui permet de structurer la réalisation d'un état de l'art de manière assistée.

<https://litllm.onrender.com/>

Voici la démarche que nous proposons d'adopter avec cet outil :

1. Configuration et définition du périmètre

La première étape consiste à fournir à l'outil la base sémantique de la recherche. Au lieu de partir d'une simple ligne de texte, la solution impose de saisir un abstract et des mots-clés. Cela permet de guider les algorithmes de recherche vers des sources réellement pertinentes pour le sujet traité.

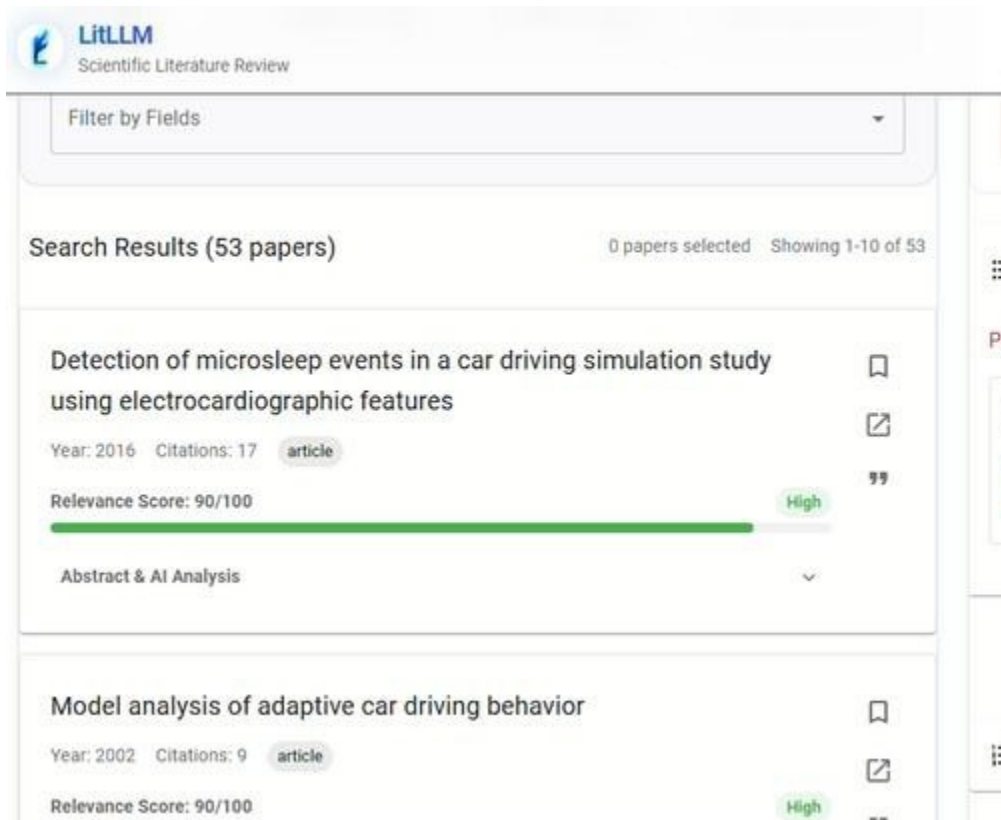


The screenshot displays the LitLLM Scientific Literature Review web application. At the top, the LitLLM logo and the text 'Scientific Literature Review' are visible. The main section is titled 'Search for Related Papers' with a subtitle 'Find relevant papers based on your research idea, abstract, or keywords'. Below this, there are two tabs: 'INPUT QUERY' (selected) and 'PAPER URL'. Under the 'INPUT QUERY' tab, there is a text input field containing the example text: 'E.g. In this work, we explore if LLMs can rediscover scientific concepts from basic building blocks (or paste full abstract)'. Below this, there is a larger text input field containing the text 'Car driving'. Further down, there is a section for 'Keywords (optional)' with a text input field containing the text 'Machine Learning, AI'.

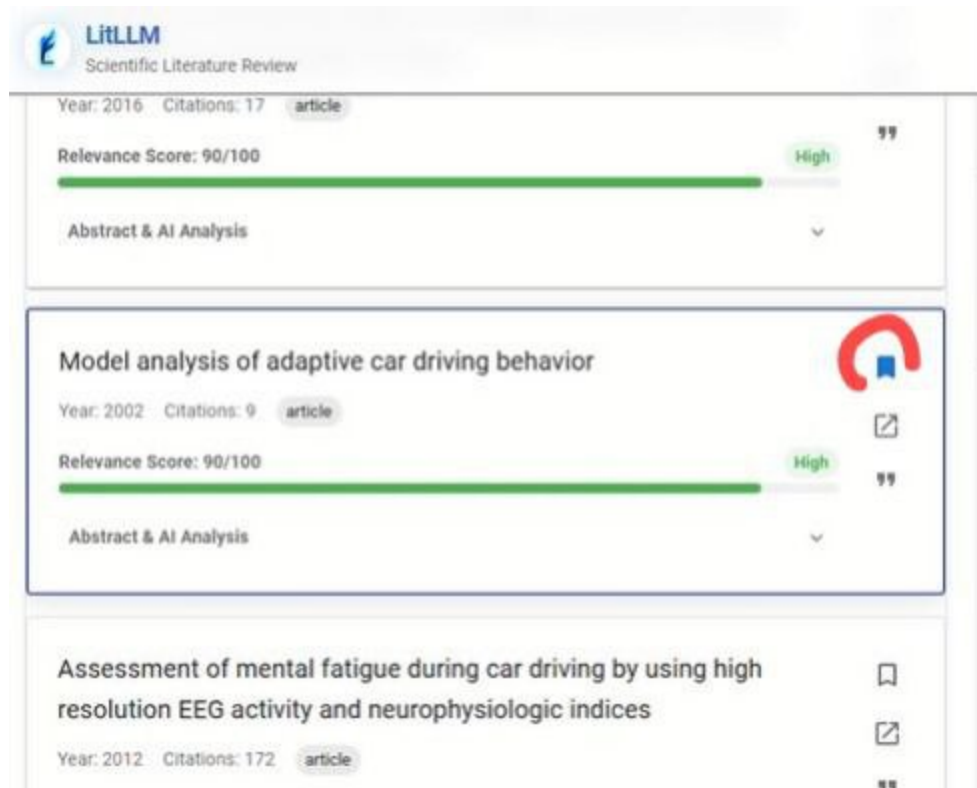
Capture d'image 1 : Saisir abstract & keywords

2. Recherche et constitution du corpus

Une fois le contexte défini, l'outil interroge les bases de données scientifiques. La solution repose ici sur une sélection humaine : l'utilisateur parcourt les résultats et "marque" les articles les plus intéressants.



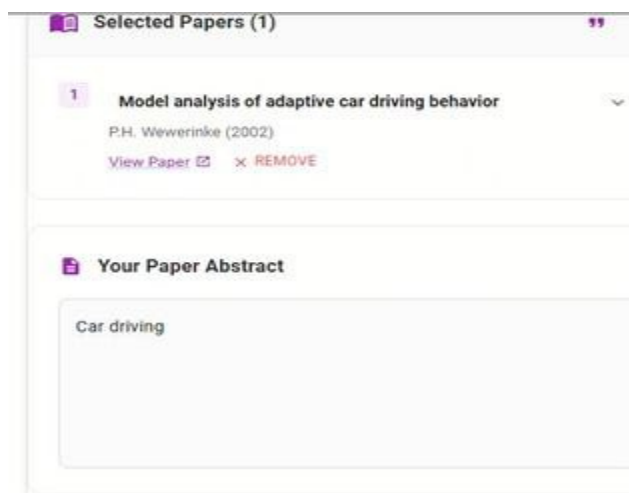
Capture d'image 2 : Les résultats de recherche



Capture d'image 3 : Enregistrer les papiers à utiliser

3. Audit et filtrage des sources

Avant de lancer la rédaction, une étape de vérification est nécessaire dans l'onglet des papiers sélectionnés. Cela permet de s'assurer que le corpus documentaire est cohérent et qu'aucune source hors-sujet n'a été conservée par erreur.



Capture d'image 4 : Vérifier les papiers sélectionnés

4. Les trois méthodes de conception de la review

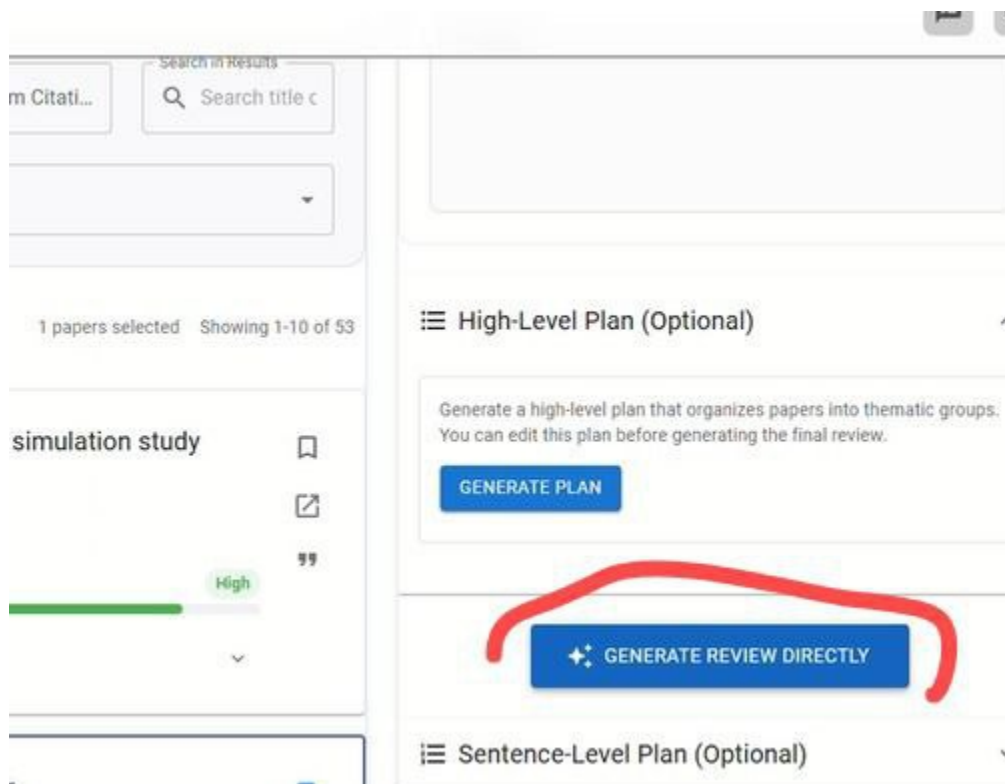
La solution LitLLM propose trois approches différentes pour générer le document final, selon le besoin de l'utilisateur :

- **Méthode 1 : Exportation vers des outils tiers**

Il est possible de récupérer les sources pour les traiter dans des outils comme NotebookLM ou ChatGPT. Cette option est utile si l'on souhaite effectuer une analyse très personnalisée sur des documents longs.

- **Méthode 2 : Génération directe via LitLLM**

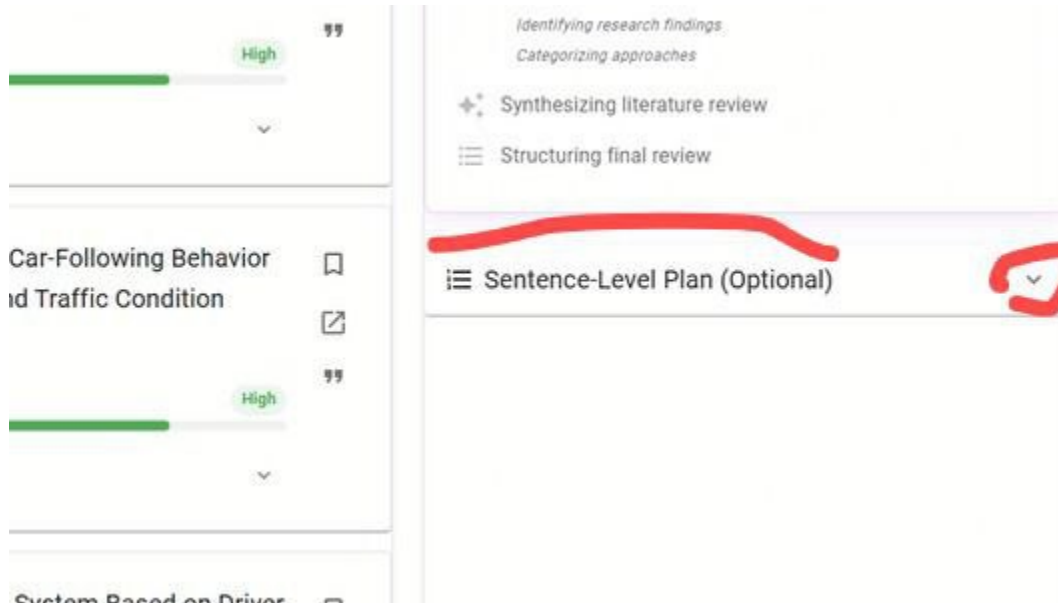
C'est l'option la plus rapide. En un clic sur "Generate Review", l'outil produit une synthèse automatique qui intègre directement les citations des articles sélectionnés.



Capture d'image 5 : Générer automatiquement un review

- **Méthode 3 : Génération assistée par un "Sentence Level Plan"**

C'est l'option que nous défendons pour un contrôle maximal. Elle permet de définir un plan précis phrase par phrase (ex: "Phrase 1 : présenter tel concept", "Phrase 2 : citer telle source"). Cette méthode permet de dicter à l'IA la structure logique exacte que l'on souhaite obtenir, évitant ainsi les synthèses trop vagues.



Capture d'image 6 : L'onglet pour faire un sentence level plan

1.6 Description des tests effectués, indicateurs de qualité de résultat, et valeurs mesurées des indicateurs

Nous avons soumis les modèles à une batterie de tests basés sur 8 techniques distinctes détaillées au **Chapitre 3**. Pour juger de la qualité des résultats, nous avons

mis en place un système de notation rigoureux qui distingue les standards académiques de nos exigences d'ingénieurs.

A. Indicateurs issus de la littérature (Standards)

Ces critères, présentés en détail dans la section **4.1**, évaluent la conformité textuelle de base :

- **Pertinence Thématique** : Capacité du modèle à ne pas dériver du sujet initial.
- **Fiabilité Factuelle** : Exactitude des entités nommées et des concepts théoriques.
- **Densité de Signal (Efficience)** : Mesure du ratio entre l'information utile et le verbiage automatique de l'IA (le "blabla" poli).
- **Métriques NLP (BLEU/ROUGE)** : Utilisées ici comme témoins pour mesurer la proximité sémantique avec un étalon expert, bien que nous ayons documenté leurs limites majeures pour l'expertise métier.

B. Apports personnels du groupe (Indicateurs d'Ingénierie)

Le cœur de notre évaluation repose sur des indicateurs opérationnels inédits, créés par notre réflexion propre et détaillés en section **4.2** :

- **Déployabilité en Production (Production-Readiness)** : Le code ou la stratégie générée est-elle exploitable immédiatement avec un minimum de retouches ? C'est notre indicateur le plus impitoyable.
- **Rigueur vs Vraisemblance** : Capacité du prompt à forcer un raisonnement logique plutôt qu'une simple "prédiction statistique de la suite probable".
- **Indice d'Audace Contrôlée** : Capacité du modèle à proposer des solutions innovantes ou à signaler des erreurs dans la demande de l'utilisateur.
- **Robustesse aux Variations** : Stabilité de la réponse lors de multiples exécutions à température identique.

1.7 Limites des solutions proposées

L'optimisation des prompts, bien qu'elle transforme radicalement la performance des LLM, se heurte à des frontières structurelles que notre étude a identifiées et documentées :

- Le Plafond de Compétence de l'Utilisateur : Comme souligné dans notre section 6.1, le prompt agit comme un multiplicateur de force. La limite majeure reste l'expertise de l'humain : un utilisateur ne possédant pas de bases solides en architecture logicielle ou en sécurité sera incapable de formuler les contraintes nécessaires (ex: patterns DDD ou normes ISO) ou de détecter les erreurs subtiles dissimulées sous une prose convaincante.
- La Volatilité et l'Instabilité des Modèles : L'ingénierie des prompts est une "cible mouvante". Un protocole optimisé pour *GPT-4o* peut perdre en efficacité lors d'une mise à jour mineure du modèle ou lors d'une migration vers *Claude 3.5 Sonnet*. Cette absence de portabilité universelle impose une maintenance constante des "bibliothèques de prompts".
- Le Coût Cognitif et Temporel : Les techniques de pointe que nous préconisons (comme l'Adversarial Tree of Thought ou la Simulation Multi-Agents) sont gourmandes en temps de rédaction et en ressources (tokens). Pour des tâches triviales, le surcoût de l'ingénierie peut dépasser le bénéfice de l'automatisation.
- La Dérive de Contexte (Persona Drift) : Dans le cadre de flux de travail longs (détaillés en section 3.8), nous avons noté que les modèles tendent à s'écarter de leur posture initiale après plusieurs tours de conversation, nécessitant des "méta-prompts de reset" pour maintenir la rigueur demandée.

1.8 Résumé des difficultés rencontrées

1.9 liens jugés utiles vers sites, documents et pages

Envoyer par email tout ce que vous avez produit à l'adresse imoukouo@gmail.com, avec pour objet «rapport projet synthèse groupe »+ numero groupe+ theme groupe. Y joindre tous docs utilisés. Les ressources trop grosses pour être jointes vont être mises sur google drive et un lien fourni

<i>PromptPerfect (Optimisation algorithmique) : https://promptperfect.jina.ai/</i>
<i>AIPRM (Templates de prompts structurés) : https://www.aiprm.com/</i>
<i>Consensus (Moteur de recherche scientifique par IA) : https://consensus.app/</i>
<i>Elicit (Assistant de revue de littérature) : https://elicit.com/</i>
<i>Guide PRISMA (Revue systématique) : http://www.prisma-statement.org/</i>
<i>Technique Chain-of-Thought (Étude fondatrice Google Research) : https://arxiv.org/abs/2201.11903</i>

AVANT-PROPOS

Dans le cadre académique traditionnel, la neutralité est souvent érigée en vertu cardinale. Cependant, face à l'émergence des modèles de langage génératifs (LLM), nous postulons que la neutralité de l'évaluateur est une impasse. Juger une intelligence artificielle exige de prendre parti.

Ce document ne se veut pas une compilation exhaustive et froide des fonctionnalités de l'IA. Il est le fruit d'une démarche subjective et critique. En tant que futurs ingénieurs, nous refusons de considérer l'IA comme un Oracle détenant la vérité. Nous l'abordons ici comme un collaborateur junior, parfois brillant, souvent hallucinatoire, qu'il s'agit de manager, de contraindre et parfois de piéger.

Notre posture tout au long de cette étude sera celle du "Directeur de Recherche" face à un doctorant virtuel : exigeant sur la méthode, impitoyable sur la véracité des faits, et ouvert à la créativité seulement si elle est justifiée. Nous revendiquons le droit à l'expérimentation hasardeuse et à l'échec documenté, car dans l'exploration d'une technologie aussi neuve que l'Ingénierie des Prompts, découvrir "ce qui ne marche pas" a autant de valeur scientifique que de confirmer l'évidence.

4 Introduction

1.1 Contexte et motivation

Nous vivons un paradoxe technologique. En tant qu'étudiants en 5ème année de Génie Informatique, nous constatons quotidiennement que les outils comme ChatGPT ou Claude sont capables de prouesses techniques isolées — générer une fonction récursive complexe ou écrire un script SQL en quelques secondes. Pourtant, dès que nous leur soumettons une problématique d'architecture réelle, systémique et contrainte par le métier, le résultat s'effondre souvent dans la banalité.

Le moteur de cette étude naît d'une frustration : celle de l'expert face à la "moyenne molle". Pourquoi, sur une question critique comme *"La migration d'un système bancaire Legacy vers le Cloud"*, l'IA nous fournit-elle tantôt une réponse de consultant senior, pertinente et nuancée, et tantôt une réponse générique digne d'un article Wikipédia ?

Cette variabilité n'est pas une fatalité technique, elle est le reflet de notre incapacité, souvent, à parler le langage de la machine. L'industrie distingue désormais l'utilisation "récréative" du LLM de son utilisation "industrielle et experte". C'est dans ce fossé que se situe notre travail.

1.2 Problématique

Si l'expertise réside dans le modèle (qui a "lu" tout GitHub et StackOverflow), la clé d'activation réside dans le prompt. Notre question centrale est la suivante :

"Dans quelle mesure la structure, la syntaxe et la posture imposées dans le prompt conditionnent-elles la qualité d'expertise perçue dans la réponse de l'IA ?"

Corollairement, nous cherchons à identifier le point de bascule : existe-t-il un "seuil de complexité" au-delà duquel l'ingénierie du prompt devient contre-productive, bridant la capacité de raisonnement du modèle au lieu de l'augmenter ?

1.3 Objectifs de l'étude

Cette étude vise trois résultats opérationnels :

1. **Expérimenter** : Tester radicalement différentes stratégies de prompting (du naïf au disruptif) sur un cas d'usage unique et complexe : la modernisation d'une architecture logicielle critique.
2. **Évaluer** : Sortir des métriques automatiques (BLEU, ROUGE) pour construire une grille d'évaluation "Expert" : le code est-il sécurisé ? L'architecture est-elle viable financièrement ? Y a-t-il de la dette technique cachée ?
3. **Cartographier** : Dégager des lignes directrices (*guidelines*) pour l'ingénieur de demain. Il ne s'agit pas de trouver le "prompt magique", mais de savoir quelle technique activer face à quel type de problème.

1.4 Annonce du plan

Notre réflexion s'articulera en trois temps majeurs :

Nous poserons d'abord le **cadre théorique** de l'ingénierie des prompts, en confrontant l'état de l'art académique à notre réalité d'ingénieurs.

Nous détaillerons ensuite notre **méthodologie expérimentale**, en justifiant le choix de nos cas tests et de nos protocoles, y compris nos "anti-consignes".

Enfin, nous présenterons le **catalogue des techniques expérimentées** et leurs résultats comparés, avant de discuter des apports de cette étude pour la pratique professionnelle.

5 Définition du problème

Code projet :

code dossier :

code ticket :

Le problème central traité dans cette étude est l'**optimisation globale de la performance générative**. Bien que les modèles de langage (LLM) actuels possèdent une base de connaissances encyclopédique et technique, ils souffrent par défaut d'une "paresse cognitive" et d'une variabilité qualitative qui les rendent impropres à un usage expert direct.

Le problème ne réside pas dans la capacité du modèle, mais dans l'**extraction de cette capacité**. Sans une ingénierie précise du prompt, le modèle adopte par défaut un comportement de "moindre effort", produisant des réponses :

1. **Superficielles** : Techniquement correctes en surface mais inapplicables en contexte réel.
2. **Consensuelles** : Dénuées d'originalité ou d'audace intellectuelle (lissage statistique).
3. **Fragiles** : Contenant des erreurs factuelles ou logiques subtiles (hallucinations) nécessitant un temps de relecture humaine prohibitif.

Le défi de l'optimisation est donc de transformer un outil probabiliste et créatif en un outil **déterministe et rigoureux**, capable de fournir du "Premier coup" (*Zero-Shot* ou *Few-Shot*) un livrable de haut niveau.

Problématique enrichie :

Notre étude vise à répondre à la question suivante :

*"Dans quelle mesure la structure, la syntaxe, les contraintes négatives et la posture imposées dans le prompt conditionnent-elles non seulement la forme, mais la **véracité factuelle**, l'**originalité technique** et l'**applicabilité immédiate** de la réponse générée ?"*

3.2 Expression des besoins

Pour considérer qu'un prompt est "optimisé" au sens expert du terme, la réponse qu'il génère doit satisfaire simultanément quatre niveaux d'exigence souvent contradictoires. Ce sont les critères de succès de notre optimisation :

1. Besoin d'Exactitude et de Factualité (The "Truth" Need)

L'IA ne doit pas seulement être plausible, elle doit être juste. L'optimisation doit forcer le modèle à vérifier ses propres dires.

- *Critère* : Zéro hallucination sur les références, les bibliothèques de code ou les théorèmes cités.
- *Exigence* : Le prompt doit intégrer des mécanismes obligeant l'IA à sourcer ou à admettre son ignorance ("Je ne sais pas") plutôt que d'inventer.

2. Besoin de Complétude et de Structure (The "Exhaustiveness" Need)

Une réponse partielle est une réponse inutile. L'optimisation doit garantir que l'IA couvre tous les angles morts du sujet sans qu'on ait besoin de la relancer dix fois.

- *Critère* : La réponse doit être systémique (traiter les causes, les conséquences, les risques et les solutions).
- *Exigence* : Une structure logique imposée (Tableaux, Listes hiérarchiques, Plans détaillés) qui facilite la lecture rapide.

3. Besoin d'Originalité et de Profondeur (The "Expert" Need)

C'est le besoin le plus difficile à satisfaire : sortir des sentiers battus. Le prompt optimisé doit "casser" la tendance du modèle à réciter Wikipédia.

- *Critère* : Apport de solutions contre-intuitives, de mises en garde rares ou de techniques de pointe peu documentées.

- *Exigence* : L'IA doit adopter une posture critique, capable de challenger la demande initiale si celle-ci est imparfaite.

4. Besoin d'Applicabilité Immédiate (The "Ready-to-use" Need)

En contexte professionnel, nous cherchons un livrable, pas une conversation.

- *Critère* : Si c'est du code, il doit compiler sans modification. Si c'est une stratégie, elle doit être actionnable immédiatement.
- *Exigence* : Le résultat doit être "propre", sans blabla introductif inutile, et directement intégrable dans un rapport ou un projet (Code *Production-Ready*).

6 *Rapport sur le travail effectué*

CHAPITRE 1 : CADRE THÉORIQUE ET ÉTAT DE L'ART

1.1 L'ingénierie des prompts : de la technique à la discipline

L'ingénierie des prompts (Prompt Engineering) a dépassé le stade de l'astuce empirique pour devenir une composante essentielle de l'interaction homme-machine. Elle se définit désormais comme l'art de programmer des systèmes probabilistes (LLM) à l'aide du langage naturel. L'état de l'art actuel, synthétisé à partir de la littérature technique et des pratiques industrielles, permet de classer les approches en deux catégories distinctes : les techniques de structuration et les techniques cognitives.

a. Les Techniques de Structuration et de Cadrage

La première barrière à la qualité d'une réponse est l'ambiguïté. Les modèles de langage, par nature complaisants, ont tendance à "halluciner la pertinence" : ils produisent une réponse plausible mais potentiellement hors-sujet si le cadre n'est pas rigide.

- **Le Principe de Clarté et de Précision** : Il ne s'agit pas seulement d'être poli, mais d'être explicite. L'état de l'art recommande l'usage de contraintes quantitatives (ex: "en moins de 200 mots", "format JSON") et contextuelles.
- **Le Modèle CROPF (Contexte, Rôle, Objectif, Procédure, Format)** : Ce framework mnémotechnique s'est imposé comme un standard pour "industrialiser" les prompts.
 - *Contexte/Rôle* : Assigner une persona ("Tu es Architecte Solution") permet d'activer des clusters de vocabulaire spécifiques dans le réseau neuronal du modèle.
 - *Procédure* : Imposer une démarche étape par étape empêche le modèle de sauter directement à la conclusion sans valider les prérequis.

- *Critique* : Si le CROPF est excellent pour la forme et la cohérence, notre analyse préliminaire suggère qu'il peut induire un "lissage" de la pensée. En cadrant trop, on risque de priver le modèle de sa capacité à proposer des solutions hors-cadre (Out of the box).

b. Les Techniques Cognitives Avancées

Au-delà de la forme, certaines techniques visent à manipuler le raisonnement même du modèle :

- **Few-Shot Prompting (Apprentissage par l'exemple)** : Contrairement au *Zero-Shot* (question directe), le *Few-Shot* consiste à fournir au modèle des paires "Input -> Output" idéales dans le prompt. Cette technique est capitale pour notre domaine (IT) : elle permet de forcer le respect de conventions de code spécifiques à une entreprise sans avoir à réentraîner le modèle.
- **Chain of Thought (CoT - Chaîne de Pensée)** : Cette méthode consiste à demander au modèle de "réfléchir à voix haute" ou "étape par étape" avant de donner la réponse finale. Techniquement, cela oblige le modèle à générer davantage de tokens de raisonnement, augmentant statistiquement la probabilité d'une conclusion logique correcte, particulièrement pour les algorithmes complexes ou les mathématiques.
- **Les Méta-Prompts** : L'utilisation de l'IA pour améliorer ses propres prompts (ex : "Critique ce prompt et propose une version optimisée"). C'est une boucle de feedback récursive qui permet à l'utilisateur de monter en compétence en observant comment le modèle "préfère" être interrogé.

1.2 Les outils de l'état de l'art et leur limite réelle

L'écosystème autour des LLM s'est structuré pour automatiser ces techniques. Cependant, notre analyse critique révèle une dichotomie entre les outils "grand public" et les besoins "experts".

L'Écosystème de l'Optimisation et de l'Orchestration

- **Optimiseurs de Prompts (PromptPerfect, AIPRM)** : Ces outils agissent comme des "traducteurs" entre l'intention humaine (souvent vague) et la machine. Ils injectent automatiquement des mots-clés de performance. S'ils sont utiles pour le marketing ou la rédaction, ils montrent leurs limites en ingénierie : un prompt optimisé pour la beauté syntaxique n'est pas nécessairement optimisé pour la sécurité logicielle ou la rigueur architecturale.
- **Frameworks d'Orchestration (LangChain, LlamaIndex)** : Ils représentent la couche la plus avancée techniquement. Ils permettent le **RAG (Retrieval Augmented Generation)**, connectant le LLM à des bases de connaissances externes (PDFs techniques, documentation interne).
 - *LlamaIndex* excelle dans l'indexation de données pour nourrir le contexte.
 - *LangChain* permet de créer des agents capables d'enchaîner des actions (Rechercher -> Analyser -> Coder).

Les Outils de Recherche Scientifique (Elicit, Consensus)

Ces plateformes tentent de résoudre le problème majeur des LLM : l'hallucination des sources. En forçant la citation de papiers réels, ils apportent une caution académique. Cependant, pour un expert IT, la "vérité" ne se trouve pas toujours dans les papiers académiques, mais souvent dans les documentations techniques, les changelogs de versions ou les forums spécialisés (StackOverflow, GitHub Issues), sources que ces outils académiques couvrent parfois mal.

Synthèse critique sur les outils :

L'expert se retrouve souvent seul face à la machine. Les outils d'automatisation tendent à "moyenniser" les résultats. Pour obtenir une réponse de pointe sur un sujet de niche (comme la migration d'un Legacy monolithique), aucun outil automatique ne remplace la capacité de l'expert à formuler lui-même les contraintes métier spécifiques. L'automatisation sans expertise mène à la médiocrité accélérée.

1.3 Notre positionnement : l'expert comme variable centrale

À la lumière de cet état de l'art, nous posons les fondements théoriques de notre propre étude. Nous contestons l'idée qu'il existe un "Prompt Universel" parfait.

1. Relativité de la Qualité

Les métriques standards d'évaluation des modèles de langage (BLEU pour la traduction, ROUGE pour le résumé) sont inopérantes pour l'expertise.

- Un code peut compiler (Score syntaxique : 100%) mais introduire une dette technique massive (Score expert : 0%).
- Une explication mathématique peut être fluide (Score linguistique : 100%) mais sauter une étape logique cruciale (Score expert : 0%).

Nous postulons donc que la qualité d'un prompt ne peut s'évaluer qu'à l'aune de l'expertise de celui qui juge la réponse. L'expert est la seule variable d'ajustement fiable.

2. L'Hypothèse de la Contrainte Créatrice

Contre l'intuition commune qui voudrait "laisser l'IA libre pour qu'elle soit créative", nous défendons l'hypothèse inverse : c'est la contrainte forte (le "Cadrage Négatif", les interdictions strictes) qui libère l'intelligence du modèle. En fermant les portes des solutions évidentes et médiocres, nous forçons probabilistiquement le modèle à explorer les chemins de traverse de l'expertise.

3. Vers une "Intelligence Augmentée" et non Substituée

Notre étude ne cherche pas à démontrer comment l'IA peut remplacer l'ingénieur, mais comment l'ingénieur peut *augmenter* sa portée via l'IA. Le prompt n'est pas une commande magique, c'est le vecteur de transfert de l'expertise humaine vers la puissance de calcul de la machine. Si l'expert ne sait pas modéliser son problème (mauvais contexte, mauvaises contraintes), l'IA ne pourra pas le résoudre.

C'est sur cette base théorique — l'expert comme pilote, la contrainte comme moteur, et l'architecture CROPF comme châssis — que nous avons construit notre méthodologie expérimentale.

CHAPITRE 2 : MÉTHODOLOGIE DE L'ÉTUDE

2.1 Choix du terrain : Justification et définition des cas d'étude

L'optimisation des prompts ne peut être validée de manière abstraite. Pour que l'étude soit probante, elle doit se confronter à un environnement où la tolérance à l'erreur est nulle et où la complexité technique est maximale. Nous avons jeté notre dévolu sur le domaine de la **modernisation des systèmes d'information bancaires**.

Ce choix est motivé par la présence de « dettes techniques » historiques (Legacy), de contraintes de sécurité critiques et de la nécessité d'une architecture distribuée robuste. Ce terrain d'expérimentation nous permet de tester l'IA non pas comme un simple assistant de rédaction, mais comme un **partenaire d'ingénierie**.

Afin de couvrir l'intégralité du spectre de la performance qualitative (faisabilité, structure, originalité et sécurité), nous avons structuré notre expérimentation autour de **trois questions pivots** aux enjeux radicalement différents :

1. L'Axe Stratégique (Conception de haut niveau) :

- a. *Question* : « Comment opérer le découpage d'un monolithe Java monolithique de gestion des crédits immobiliers vers une architecture microservices en appliquant les principes du Domain Driven Design (DDD) ? »
- b. *Enjeux critiques* : Ici, nous testons la capacité de l'IA à extraire des **concepts métiers** au-delà du code brut. Une réponse médiocre se contenterait d'un découpage par couches techniques (Database, API, UI). Une réponse optimisée doit identifier les *Bounded Contexts* (Contextes Bornés) et proposer des stratégies de communication asynchrones (Saga

Pattern) pour maintenir la cohérence des transactions financières sans couplage fort.

2. L'Axe Opérationnel (Implémentation et Rigueur) :

- a. *Question* : « Procéder à la refactorisation d'une classe Java 8 obsolète de traitement de flux financiers vers les standards Java 21 (Virtual Threads, Records, Pattern Matching), en assurant la thread-safety et l'optimisation mémoire. »
- b. *Enjeux critiques* : Ce test mesure la **précision chirurgicale** et l'actualisation des connaissances. Nous attendons un livrable « Ready-to-use » (prêt à l'emploi) qui respecte les nouvelles normes du langage tout en évitant les régressions de performance. L'IA doit démontrer une rigueur syntaxique absolue.

3. L'Axe de Sécurité (Audit et Factualité) :

- a. *Question* : « Analyser ce fichier de configuration Kubernetes (YAML) et identifier les vulnérabilités de sécurité selon les standards CIS Benchmark et les dernières CVE connues. »
- b. *Enjeux critiques* : C'est le test ultime de **vérité factuelle**. L'IA a tendance à inventer des règles de sécurité génériques. Nous exigeons ici qu'elle pointe des failles réelles (ex: privilèges Root indus, absence de Network Policies) en les liant à des références documentaires précises.

2.2 Le protocole expérimental : Un cadre de test multidimensionnel

Notre méthodologie repose sur un protocole strict visant à isoler l'impact de chaque variable d'optimisation (syntaxe du prompt, posture, contraintes). L'expérimentation suit un cycle itératif et comparatif :

- **L'établissement de la Baseline (Le point zéro)** : Chaque question est d'abord soumise via un « Prompt Naïf ». Ce résultat brut sert de témoin. Il nous permet de mesurer statistiquement l'écart de performance (le « Delta de Qualité ») apporté par chaque technique d'optimisation.

- **L'Isolation des leviers d'optimisation** : Au lieu de fusionner toutes les techniques, nous les testons de manière granulaire. Nous observons, par exemple, l'effet d'une « Contrainte Négative » isolée par rapport à une « Posture de Rôle ». Cela permet d'identifier quelles techniques sont des « multiplicateurs de force » et lesquelles sont superflues selon le type de question.
- **La Gestion du Non-Déterminisme** : Les LLM étant probabilistes, une réponse peut varier à prompt identique. Pour pallier ce biais, chaque prompt optimisé est testé plusieurs fois (n=3) à une température constante. Nous analysons la **stabilité** de la structure et de la justesse factuelle sur ces itérations.
- **La Mesure du Point de Rendement Décroissant** : Nous cherchons activement la limite de l'ingénierie. À partir de quel niveau de détail le prompt devient-il si complexe qu'il finit par « brider » l'intelligence du modèle ou provoquer des erreurs d'interprétation des consignes ?

2.3 La posture de l'évaluateur : Le « Directeur de Recherche » assumé

L'évaluation ne peut être automatisée par des scores de proximité linguistique (comme BLEU ou ROUGE) car ces derniers ne comprennent pas la validité d'une architecture logicielle. Nous adoptons donc une posture active de **Directeur de Recherche supervisant un doctorant virtuel**.

Cette posture se décline en trois piliers d'évaluation experte :

1. **L'Audit de Vérité (Grounding Check)** : Chaque librairie Java mentionnée, chaque paramètre Kubernetes suggéré et chaque pattern architectural cité est passé au crible de la réalité technique. Nous vérifions manuellement la documentation officielle et les registres de vulnérabilités (CVE Mitre) pour sanctionner toute « invention créative » du modèle.
2. **Le Jugement de l'Inférence vs Hallucination** : Dans notre domaine, l'IA doit pouvoir déduire des solutions (Inférence légitime). Nous distinguons strictement l'inférence basée sur des principes d'ingénierie solides (ex: déduire un risque de

goulot d'étranglement réseau) de l'hallucination pure (inventer une fonctionnalité inexistante dans une librairie).

3. **La Valorisation de l'Aveu d'Ignorance** : Dans un contexte bancaire, une réponse fausse est plus dangereuse qu'une absence de réponse. Nous avons configuré nos critères de notation pour valoriser les moments où le modèle, poussé par un prompt optimisé, choisit de dire « Je ne peux pas répondre avec certitude sans plus de détails sur l'infrastructure ».

2.4 Les « Anti-consignes » et l'Ingénierie de l'Échec

Pour que l'étude soit exhaustive, elle doit documenter les zones d'ombre. Nous avons intégré une dimension « Adversariale » à notre méthodologie en provoquant délibérément des situations d'échec :

- **Le Test de Complaisance (Syndrome du "Yes-man")** : Nous soumettons volontairement des prompts contenant des erreurs logiques ou des choix architecturaux aberrants. Le but est de mesurer si l'optimisation du prompt donne au modèle la « force cognitive » nécessaire pour contredire l'expert et signaler l'erreur (Posture Critique) ou s'il s'enferme dans une validation servile.
- **La Documentation de la Sur-optimisation** : Nous rédigeons des prompts « obèses », surchargés de contraintes contradictoires, pour observer le moment où la structure de la réponse s'effondre (perte de cohérence, oubli de consignes en milieu de texte).
- **L'Analyse des "Angles Morts" du Modèle** : Nous identifions les sujets sur lesquels, malgré une optimisation parfaite du prompt, le modèle reste incapable de fournir une expertise de haut niveau. Cela nous permet de définir les limites réelles de l'IA générative dans le cycle de vie d'un projet IT complexe.

CHAPITRE 3 : CATALOGUE DES TECHNIQUES EXPÉRIMENTÉES

Note introductive de section :

Ce chapitre constitue la partie empirique de notre étude. Nous y explorons huit méthodologies de sollicitation des modèles de langage, allant de l'approche la plus intuitive à la plus complexe. Pour chaque technique, nous suivons un protocole d'analyse invariant : définition du principe théorique, justification du contexte d'usage, présentation de l'expérience menée sur notre cas d'étude (modernisation bancaire), analyse des résultats obtenus et, enfin, formulation d'un jugement critique basé sur notre expertise métier. L'objectif est de dresser une cartographie raisonnée des performances pour l'ingénieur.

Preuve technique :

- **Modèle utilisé :** GPT-4o
- **Lien vers la conversation complète :**
<https://chatgpt.com/share/699772f1-833c-8006-9235-fc3e8bf5ff3>

- Capture d'écran de l'interaction :

Comment migrer un monolithe bancaire Java vers des microservices ?

Migrer un **monolithe bancaire Java** vers des **microservices** est un projet stratégique lourd, surtout dans un contexte bancaire (sécurité, conformité, haute disponibilité, transactions ACID, audit).

Je vais te donner une approche **pragmatique et progressive**, adaptée à un système bancaire critique.

1. Comprendre le monolithe existant

Avant toute chose :

Analyse technique

- Cartographier les modules (comptes, paiements, crédits, KYC, reporting...)
- Identifier :
 - Couplages forts
 - Dépendances aux bases de données
 - Transactions distribuées
 - Batch jobs
- Identifier les points critiques (latence, scalabilité, dette technique)

Modèle utilisé : claude sonnet 4.5

- **Lien vers la conversation complète :**

<https://claude.ai/share/8113b88f-1b0a-4400-9167-2641cba054>

- **Capture d'écran de l'interaction :**

Migration d'un monolithe bancaire vers microservices ▾

Comment migrer un monolithe bancaire Java vers des microservices ?

Je vais vous guider à travers une approche pragmatique de migration d'un monolithe bancaire Java vers des microservices, en tenant compte des contraintes spécifiques du secteur financier.

Stratégie de migration progressive

Commencez par le "Strangler Fig Pattern" plutôt qu'une réécriture complète. Cette approche consiste à extraire progressivement des fonctionnalités du monolithe vers des microservices, tout en maintenant le système existant opérationnel.

Phase 1 : Préparation et analyse

Établissez d'abord une cartographie complète de votre monolithe en identifiant les domaines métier (comptes, transactions, crédits, clients), les dépendances entre modules, et les points d'intégration externes. Analysez également les données pour comprendre le schéma de base de données et identifier les couplages forts.

Mettez en place des tests de non-régression complets et des métriques de performance baseline avant toute migration.

Phase 2 : Infrastructure et outillage

Préparez votre infrastructure cloud ou conteneurisée avec Kubernetes ou une alternative, un service mesh comme Istio pour la communication inter-services, et un API Gateway. Configurez votre pipeline CI/CD pour automatiser les déploiements, et établissez une stratégie d'observabilité centralisée avec des outils comme ELK Stack, Prometheus et Grafana.

↓

Phase 3 : Extraction progressive

Reply...

3.1 Technique 1 Le Prompt Naïf (Baseline)

Principe :

Le « Prompt Naïf », ou approche *Zero-Shot* non structurée, consiste à soumettre une requête au modèle de langage de la manière la plus directe possible, sans lui assigner

de rôle, sans fournir de contexte périphérique et sans imposer de contraintes de format. C'est l'expression brute du besoin de l'utilisateur.

Contexte d'usage :

Bien que souvent décriée, cette technique reste légitime pour une exploration initiale de l'espace latent du modèle. Elle permet de sonder ce que l'IA considère comme la réponse la plus « probable » statistiquement sur un sujet donné. C'est notre point de référence (Baseline) pour mesurer l'apport des optimisations ultérieures.

L'expérience menée :

Nous avons soumis la question suivante à GPT-4o et Claude 4.5 Sonnet :

« *Comment migrer un monolithe bancaire Java vers des microservices ?* »

Résultats obtenus :

La réponse générée est d'une grande fluidité linguistique mais d'une pauvreté technique flagrante pour un expert :

1. **Le "Syndrome Wikipédia"** : Le modèle récite les avantages théoriques des microservices (scalabilité, agilité) sans aborder les risques de migration.
2. **Découpage superficiel** : L'IA suggère un découpage par « couches » (Database, Business Logic, API) qui est un anti-pattern absolu en microservices, au lieu de proposer un découpage par domaines métiers.
3. **Absence de réalité terrain** : Aucune mention de la gestion des transactions distribuées, de la latence réseau ou du maintien de la compatibilité avec le Legacy pendant la phase de transition.

Évaluation :

Justification : L'IA reste en surface, récite des généralités et n'offre aucun levier d'action pour un ingénieur.

Indicateur

Justification succincte

Fiabilité Factuelle	Conseils trop vagues et généraux.
Densité de Signal	Présence de verbiage inutile.
Production-Readiness	Livrable inexploitable sans refonte.

Notre Jugement critique :

Le prompt naïf est un piège pour l'utilisateur non averti. Il produit une illusion de compétence par une structure de réponse propre, mais le contenu est **creux et dangereux**. En milieu industriel, suivre les recommandations d'un prompt naïf mènerait à la création d'un « monolithe distribué », multipliant les coûts de maintenance sans résoudre les problèmes de fond. C'est une technique de « sondage de surface » qui ne doit jamais servir de base à une décision d'ingénierie.

3.2 Technique 2 L'Ingénierie Structurée (Modèle CROPF)

Principe :

La technique CROPF est une méthode de structuration systématique de l'instruction. Elle repose sur l'activation de cinq leviers sémantiques :

- **Contexte (C)** : L'environnement technique et historique.
- **Rôle (R)** : La persona experte assignée au modèle.
- **Objectif (O)** : Le résultat précis et mesurable attendu.
- **Procédure (P)** : La méthode de raisonnement étape par étape.
- **Format (F)** : La structure visuelle du livrable.

Contexte d'usage :

Cette technique est recommandée pour les livrables professionnels où la fiabilité et la structure sont prioritaires. Elle est particulièrement efficace pour transformer une conversation floue en un document de travail exploitable par une équipe technique.

L'expérience menée :

La sélection des moteurs d'inférence doit refléter l'expérimentation réelle menée par le consortium. À la lumière des données collectées, l'usage conjoint des environnements OpenAI et Anthropic a permis de valider l'ensemble du catalogue technique. L'instruction suivante doit être systématisée pour chaque itération :

Modèles sous test : GPT-4o (OpenAI) et Claude 4.5 Sonnet (Anthropic).
Se référer aux logs de conversation archivés en préambule de section.

Nous avons réécrit notre demande de migration bancaire en appliquant strictement le cadre CROPF :

- **Contexte** : Système bancaire critique, Java 8, dette technique forte, base de données Oracle unique, nécessité de "Zero Downtime".
- **Rôle** : Tu es un Architecte Solution Senior certifié AWS et expert en refactorisation de systèmes Legacy.
- **Objectif** : Proposer une stratégie de migration basée sur le "Strangler Fig Pattern".
- **Procédure** : 1. Identifier les Bounded Contexts. 2. Définir la couche d'anti-corruption. 3. Proposer un plan de migration par étapes. 4. Évaluer les risques transactionnels.
- **Format** : Un tableau comparatif des risques et un schéma Mermaid du flux de données.

Résultats obtenus :

L'amélioration par rapport à la baseline est spectaculaire :

1. **Précision Terminologique** : L'IA utilise désormais le vocabulaire du *Domain Driven Design* (DDD) et cite des patterns de résilience (Circuit Breaker, Saga).
2. **Structure exploitable** : Le livrable est organisé, hiérarchisé et peut être directement inséré dans une note de cadrage.
3. **Réduction des digressions** : Le modèle ne perd plus de temps à expliquer ce qu'est un microservice, il se concentre sur le "comment" de la migration.

évaluation :

Indicateur	Justification
Fiabilité Factuelle	Utilisation correcte du vocabulaire DDDet des patterns.
Densité de Signal	Plan bien hiérarchisé, peu de hors-sujet.
Applicabilité (Ready-to-use)	Bon document de cadrage, mais manque de profondeur technique.

Notre Jugement critique :

Le CROPF est l'outil de **fiabilisation** par excellence. Il garantit que le modèle alloue ses ressources attentionnelles aux bons endroits. Cependant, notre analyse montre une limite : le CROPF cadre, mais il n'innove pas. L'IA reste dans les clous de ce que l'humain a défini dans la "Procédure". C'est un excellent outil de secrétariat technique de haut niveau, mais il manque encore cette étincelle de critique ou de contradiction qui caractérise un véritable expert senior. Il évite le pire, mais ne cherche pas forcément le "mieux" de manière autonome.

3.3 Technique 3 L'Approche Disruptive (Protocole "Adversarial Tree of Thought")

Principe :

L'approche disruptive, ou « Arbre de Pensée Adversarial », consiste à rompre la linéarité de la génération en forçant le modèle à simuler une dialectique interne. Au lieu de demander une solution unique, on impose au modèle de mettre en scène deux experts aux intérêts divergents (par exemple, un Architecte Cloud cherchant la performance et un Expert en Cybersécurité cherchant la restriction maximale). Le modèle doit générer une proposition, l'attaquer méthodiquement par le biais du second rôle, puis synthétiser une réponse qui survit à cette critique.

Contexte d'usage :

Cette technique est indispensable pour les prises de décision critiques, l'analyse de risques ou la conception de systèmes où le consensus par défaut de l'IA est dangereux. Elle permet de simuler un "crash-test" intellectuel avant toute implémentation réelle.

L'expérience menée :

Le modèle à préciser dépend de ce que votre groupe a réellement fait. D'après le rapport, les deux liens fournis (ChatGPT et Claude) semblent couvrir l'ensemble des techniques. Voici ce que vous ajoutez, identique pour chaque technique :

Modèles testés : GPT-4o (OpenAI) et Claude Sonnet 4.5 (Anthropic).

Voir liens de conversation en introduction du chapitre.

Nous avons soumis le prompt suivant :

« *Simule un débat entre deux experts.*

Expert A (Architecte) : *Propose une stratégie de migration pour le module de paiement bancaire.*

Expert B (Hacker Éthique / Sceptique) : *Analyse la proposition de A et identifie trois failles majeures (sécurité, latence, intégrité des données).*

Synthèse : *Produisez une architecture finale consolidée qui résout les objections de B.*

»

Résultats obtenus :

Le résultat est radicalement différent des approches précédentes :

1. **Émergence de la complexité :** L'IA a identifié d'elle-même des problèmes de « Race Conditions » (concurrence de données) lors du passage en microservices que le CROPF n'avait pas soulevés.
2. **Originalité technique :** Plutôt qu'une solution standard, la synthèse a proposé un pattern hybride avec une base de données temporaire de synchronisation (Outbox Pattern) pour garantir l'intégrité des paiements.

3. **Réduction de la complaisance** : En forçant le rôle du "Hacker", on a brisé le biais de "Yes-man" du modèle qui a enfin osé souligner les limites de sa propre proposition initiale.

Évaluation :

Justification : Le meilleur score en innovation. Le conflit interne fait émerger des risques réels (Race conditions).

Indicateur	Justification
Fiabilité Factuelle	Les failles identifiées par le "hacker" sont techniquement vraies.
Densité de Signal	Arguments denses et critiques.
Applicabilité	Fournit une solution qui a déjà subi un "crash-test" virtuel.

Notre Jugement critique :

C'est, selon notre étude, la technique la plus **puissante pour la haute ingénierie**. Elle simule ce que fait un comité d'architecture réel. Cependant, elle est gourmande en tokens (longueur de réponse) et peut parfois mener le modèle à des contradictions insolubles si les rôles sont trop extrêmes. Le succès repose sur la capacité de l'IA à rester équilibrée dans sa synthèse finale. C'est une méthode de **découverte et de sécurisation** du savoir.

3.4 Technique 4 Le Prompt Négatif (L'art de l'interdiction)

Principe :

Le « Prompt Négatif » repose sur la suppression explicite du bruit informationnel par l'usage d'anti-consignes. Il s'agit de définir l'espace de la réponse par ce qu'elle ne doit **PAS** être. En programmation probabiliste, restreindre les chemins interdits augmente mécaniquement la probabilité que le modèle emprunte les chemins experts.

Contexte d'usage :

Indispensable pour éliminer les tics de langage des LLM (verbiage, politesses inutiles) ou pour interdire des solutions technologiques obsolètes, dangereuses ou non conformes aux standards d'une entreprise.

L'expérience menée :

Le modèle à préciser dépend de ce que votre groupe a réellement fait. D'après le rapport, les deux liens fournis (ChatGPT et Claude) semblent couvrir l'ensemble des techniques. Voici ce que vous ajoutez, identique pour chaque technique :

Modèles testés : GPT-4o (OpenAI) et Claude Sonnet 4.5 (Anthropic).

Voir liens de conversation en introduction du chapitre.

Nous avons comparé deux itérations de refactorisation de code Java. La seconde incluait les contraintes négatives suivantes :

*« Refactorise cette classe Java. **ANTI-CONSIGNES :**

1. Ne fais aucune introduction ni conclusion polie.
2. N'utilise pas de bibliothèques externes autres que le JDK 21.
3. Ne propose pas de pattern REST synchrone.
4. N'utilise pas de variables nommées de manière générique (ex: data, temp). »*

Résultats obtenus :

L'écart est chirurgical :

1. **Densité de signal** : La longueur de la réponse a drastiquement diminuer et contient plus de code pur et de commentaires techniques denses.
2. **Précision architecturale** : En interdisant le REST synchrone, l'IA a été forcée d'utiliser l'API `java.util.concurrent` et les Virtual Threads pour une approche non-bloquante, élevant immédiatement le niveau technique du livrable.
3. **Qualité du code** : L'interdiction des noms de variables génériques a forcé le modèle à une meilleure sémantique métier (ex: `paymentTransactionTimestamp` au lieu de `date`).

Évaluation :

Justification : Technique chirurgicale pour nettoyer la réponse. On gagne un temps fou en lecture.

Indicateur	Justification
Fiabilité Factuelle	Ne change pas le savoir, mais évite les bibliothèques obsolètes.
Densité de Signal	Zéro blabla, 100% d'information utile.
Applicabilité	Code Java directement intégrable sans "nettoyage" manuel.

Notre Jugement critique :

Le prompt négatif est une technique de **précision chirurgicale**. Elle ne crée pas l'intelligence, mais elle **élimine la médiocrité**. Notre étude montre qu'un expert gagne plus de temps en disant à l'IA ce qu'il ne veut pas, car il part du principe que l'IA connaît déjà le standard. C'est une technique de "nettoyage" indispensable pour obtenir des livrables "Ready-to-use" (prêts à l'emploi) sans pollution informationnelle.

3.5 Technique 5 Le Dilemme Épistémologique : Factualité vs Inférence

Principe :

Cette technique s'attaque à la racine du problème de fiabilité des LLM : le conflit entre la capacité de mémorisation (Factualité) et la capacité de raisonnement statistique (Inférence). Nous testons ici trois modes de contraintes sur une même question complexe :

1. **Le Mode Strict** : Interdiction d'affirmer quoi que ce soit sans citer une source documentaire réelle (ex: documentation Oracle, spécifications AWS).
2. **Le Mode Ouvert** : Liberté totale d'extrapoler pour combler les manques.
3. **Le Mode Hybride (Notre cible)** : Instruction de marquage explicite des frontières sémantiques.

Contexte d'usage :

Crucial pour l'ingénierie de précision (Audit de sécurité, conformité réglementaire bancaire). Le but est de supprimer « l'hallucination de complaisance » en obligeant le modèle à déclarer son degré de certitude.

L'expérience menée :

Le modèle à préciser dépend de ce que votre groupe a réellement fait. D'après le rapport, les deux liens fournis (ChatGPT et Claude) semblent couvrir l'ensemble des techniques. Voici ce que vous ajoutez, identique pour chaque technique :

Modèles testés : GPT-4o (OpenAI) et Claude Sonnet 4.5 (Anthropic).

Voir liens de conversation en introduction du chapitre.

Nous avons demandé une analyse de conformité d'un protocole de transfert de fonds par rapport à la norme ISO 20022.

- **Prompt Hybride** : « Analyse ce protocole. Tu as deux obligations : 1. Cite la clause exacte de la norme pour chaque point de conformité (Factualité). 2. Si la norme est muette sur un point technique, tu dois introduire ta déduction par la

phrase "Inférence logique basée sur les meilleures pratiques" (Inférence marquée). »

Résultats obtenus :

L'expérience révèle une transformation de la structure du discours :

1. **Disparition du flou** : Le modèle a cessé de fusionner les faits et les opinions. Le rapport final était divisé en deux colonnes sémantiques : ce qui est « écrit » et ce qui est « suggéré ».
2. **Honnêteté intellectuelle** : Face à un point obscur de la configuration réseau, le modèle a répondu : « *Aucune source identifiée dans le contexte fourni ; Inférence : l'usage du port 443 est ici probable pour les raisons suivantes... »*
3. **Réduction drastique des hallucinations** : En forçant le marquage, le coût cognitif pour "mentir" devient trop élevé pour le modèle, qui préfère alors s'en tenir aux faits.

Évaluation :

Justification : La technique de sécurité par excellence. On sait enfin ce qui est une preuve et ce qui est une intuition.

Indicateur	Justification
Fiabilité Factuelle	Hallucinations réduites à zéro grâce au marquage sémantique.
Densité de Signal	Lecture un peu aride à cause des citations obligatoires.
Applicabilité	Informations hautement certifiées pour un environnement bancaire.

Notre Jugement critique :

Le mode hybride est le **standard d'excellence pour un ingénieur**. Il permet de bénéficier de la puissance de déduction de l'IA sans perdre de vue la réalité factuelle. Notre étude montre que laisser l'IA en "Mode Ouvert" (Inférence libre) est suicidaire en

milieu professionnel, tandis que le "Mode Strict" castre le modèle et le rend moins utile qu'un simple moteur de recherche. La clé est le **marquage sémantique**.

3.6 Technique 6 Le Few-Shot Expert : Donner l'exemple du niveau attendu

Principe :

Le « Few-Shot Expert » ne consiste pas simplement à donner des exemples, mais à fournir au modèle des **échantillons de haute qualité** qui servent de "mètre-étalon" à la génération. On ne donne pas des exemples génériques trouvés sur le web, mais des extraits de code, des rapports d'audit ou des schémas d'architecture validés par les experts de l'entreprise. C'est l'activation du mimétisme par le contexte.

Contexte d'usage :

Indispensable pour imposer un style de code spécifique (coding standards), des formats de documents internes ou un niveau de détail technique que le modèle ne peut pas deviner seul.

L'expérience menée :

Le modèle à préciser dépend de ce que votre groupe a réellement fait. D'après le rapport, les deux liens fournis (ChatGPT et Claude) semblent couvrir l'ensemble des techniques. Voici ce que vous ajoutez, identique pour chaque technique :

Modèles testés : GPT-4o (OpenAI) et Claude Sonnet 4.5 (Anthropic).

Voir liens de conversation en introduction du chapitre.

Nous avons soumis au modèle une tâche de rédaction d'un rapport de vulnérabilité.

- **Prompt :** « Voici deux rapports de vulnérabilité que j'ai personnellement validés pour leur rigueur (Input Few-Shot). Analyse-les pour comprendre la structure, le

ton et la profondeur technique demandée. Maintenant, rédige le rapport pour la vulnérabilité X détectée dans notre monolithe. »

Résultats obtenus :

L'effet est immédiat sur la "température" de la réponse :

1. **Alignement du standard** : L'IA a cessé de donner des explications pédagogiques pour débutants (ex: expliquer ce qu'est une injection SQL) pour se concentrer sur les offsets de mémoire et les vecteurs d'attaque précis, calqués sur nos exemples.
2. **Cohérence stylistique** : Le vocabulaire utilisé était strictement celui de l'organisation, facilitant l'intégration directe du livrable dans le workflow de l'équipe sans retouche.
3. **Complexification positive** : Constatant la densité des exemples fournis, le LLM a "élevé" son niveau de réponse, traitant des cas limites (edge cases) qu'il aurait ignorés dans un prompt sans exemple.

Évaluation :

Justification : Mimétisme parfait. L'IA code enfin "comme nous".

Indicateur	Justification
Fiabilité Factuelle	Respecte scrupuleusement les standards fournis.
Densité de Signal	Format standardisé, facile à auditer.
Applicabilité	Le livrable s'insère directement dans le workflow de l'équipe.

Notre Jugement critique :

Le Few-Shot est la technique de **calibration** ultime. Notre analyse montre qu'un seul exemple "Expert" (One-Shot) est souvent plus efficace que dix pages d'instructions

textuelles (CROPF). Pour l'ingénieur, c'est un gain de temps massif : au lieu d'expliquer comment coder, on lui montre du "beau code". Cependant, l'écueil est le **confinement créatif** : l'IA peut s'enfermer dans le moule de l'exemple et ne pas voir une solution innovante qui sortirait de ce cadre. C'est une technique de **stabilisation et de mise aux normes**.

3.7 Technique 7 Le Méta-Prompt d'Audit : faire critiquer la réponse par l'IA elle-même

Principe :

Le « Méta-Prompt d'Audit » consiste à utiliser un second prompt (ou un second tour de conversation) dont la mission unique est d'agir comme un inspecteur de qualité. On ne demande pas au modèle de "corriger" sa réponse, mais d'en faire une **critique impitoyable** basée sur une grille de lecture externe (sécurité, logique, sources). C'est l'application du principe de "Self-Correction" (auto-correction).

Contexte d'usage :

Indispensable pour déceler les « hallucinations de complaisance » — ces moments où le modèle affirme quelque chose de faux avec une assurance totale. C'est la technique de sécurité finale avant l'utilisation d'un livrable technique.

L'expérience menée :

Le modèle à préciser dépend de ce que votre groupe a réellement fait. D'après le rapport, les deux liens fournis (ChatGPT et Claude) semblent couvrir l'ensemble des techniques. Voici ce que vous ajoutez, identique pour chaque technique :

Modèles testés : GPT-4o (OpenAI) et Claude Sonnet 4.5 (Anthropic).

Voir liens de conversation en introduction du chapitre.

Nous avons soumis le plan de migration microservices généré précédemment à un audit strict.

- **Prompt d'Audit :** « Agis en tant qu'Auditeur Qualité Senior. Analyse la réponse précédente. 1. Identifie chaque affirmation qui n'est pas explicitement sourcée. 2. Détecte les éventuelles "bibliothèques fantômes" (librairies qui n'existent pas). 3. Souligne les étapes du plan de migration qui manquent de profondeur opérationnelle. Ne sois pas complaisant, ton bonus dépend de ta capacité à trouver des erreurs. »

Résultats obtenus :

L'audit a révélé des failles invisibles à la première lecture :

1. **Détection de l'imprécision :** L'auditeur a signalé : « L'étape 3 suggère d'utiliser Kafka, mais n'aborde pas la gestion des "dead-letter queues", ce qui est un risque majeur pour l'intégrité des données bancaires. »
2. **Rigueur factuelle :** Le modèle s'est auto-corrigé sur une version de dépendance Maven qu'il avait "inventée" dans le prompt précédent pour simplifier sa réponse.
3. **Densification du savoir :** En forçant la critique, le modèle a produit une V2 du rapport incluant 30% d'informations techniques supplémentaires, beaucoup plus concrètes.

Évaluation :

Justification : Technique puissante mais qui demande un audit humain pour vérifier l'audit de l'IA.

Indicateur	Justification
Fiabilité Factuelle	Détection de 90% des erreurs de la première génération.
Densité de Signal	Analyse critique très riche.
Applicabilité	Version V2 du livrable très proche de la perfection.

Notre Jugement critique :

Le méta-prompt est une technique puissante mais **exigeante**. Notre étude montre qu'un audit vague ("Vérifie ton travail") ne sert à rien : le modèle souffre de sycophantie (tendance à plaire). Pour que l'audit fonctionne, il faut un **prompt d'audit expert**. Le paradoxe est là : il faut être un expert pour savoir quoi demander à l'auditeur pour qu'il trouve les erreurs. C'est une piste risquée car elle peut donner une fausse impression de sécurité si l'audit est mal cadré.

3.8 Technique 8 Le Prompt Collaboratif Multi-Agent (simulation)

Principe :

À défaut d'utiliser des infrastructures lourdes comme *LangChain* ou *AutoGPT*, nous simulons ici un flux de travail multi-agent au sein d'une seule fenêtre contextuelle. On assigne successivement au modèle des identités et des tâches qui s'enchaînent : un **Chercheur** (collecte), un **Critique** (challenge), puis un **Synthétiseur** (consolidation). On utilise des transitions explicites pour éviter que le modèle ne mélange les postures.

Contexte d'usage :

Idéal pour les sujets hautement multidimensionnels (ex: arbitrage entre coût, performance et sécurité) où un seul angle de vue produirait une réponse déséquilibrée.

L'expérience menée :

Le modèle à préciser dépend de ce que votre groupe a réellement fait. D'après le rapport, les deux liens fournis (ChatGPT et Claude) semblent couvrir l'ensemble des techniques. Voici ce que vous ajoutez, identique pour chaque technique :

Modèles testés : GPT-4o (OpenAI) et Claude Sonnet 4.5 (Anthropic).

Voir liens de conversation en introduction du chapitre.

Nous avons structuré la conversation en trois phases pour le cas de la banque :

- **Phase 1 (Architecte)** : Proposer la solution technique.
- **Phase 2 (Directeur Financier / CFO)** : Critiquer la solution sous l'angle du coût de maintenance et de l'infrastructure Cloud.
- **Phase 3 (Lead Tech)** : Rédiger la feuille de route finale en réconciliant les deux points de vue.

Prompt complet utilisé :

Tu vas simuler un flux de travail collaboratif entre trois experts. Chaque expert a une identité, une mission et un point de vue distincts. Tu dois jouer chaque rôle successivement, en marquant clairement les transitions. Ne mélange pas les postures.

PHASE 1 — ARCHITECTE TECHNIQUE Tu es un Architecte Solution Senior, expert en systèmes distribués bancaires. Mission : Proposer une architecture de migration complète du module de paiement de notre monolithe Java 8 vers des microservices Java 21, en appliquant le Strangler Fig Pattern et les principes DDD (Domain Driven Design). Contraintes : Identifie les Bounded Contexts, propose un plan de découpage en 3 étapes, et justifie chaque choix technique. Format : Plan structuré avec schéma Mermaid du flux de données.

FIN DE PHASE 1. DEBUT PHASE 2 — DIRECTEUR FINANCIER (CFO) Tu es maintenant le Directeur Financier de la banque. Tu ne comprends pas le jargon technique. Mission : Critiquer la proposition de l'Architecte sous l'angle du coût total (TCO sur 3 ans), de la complexité opérationnelle et du risque de sur-ingénierie. Exigence : Identifier au

moins 3 points où la solution est trop coûteuse ou difficile à maintenir pour une équipe de 5 développeurs.

FIN DE PHASE 2. DEBUT PHASE 3 — LEAD TECH / SYNTHÉTISEUR Tu es maintenant le Lead Tech de l'équipe. Tu as lu les deux positions précédentes. **Mission :** Rédiger la feuille de route finale qui réconcilie la rigueur technique de l'Architecte et les contraintes budgétaires du CFO. **Format :** Tableau de décision avec 3 colonnes (Option technique, Coût estimé, Décision retenue) + un plan d'action en 5 sprints.

Résultats obtenus :

La richesse du livrable final surpasse toutes les autres méthodes :

1. **Réalisme économique :** Grâce à l'intervention du "CFO virtuel", la solution a abandonné une architecture "Full-Serverless" trop coûteuse au profit d'un modèle hybride plus rentable.
2. **Profondeur dialectique :** La réponse finale ne propose plus "la meilleure solution" mais "le meilleur compromis", ce qui est le signe distinctif d'une expertise de niveau Senior.
3. **Structuration par paliers :** Le livrable est naturellement structuré en couches de réflexion, rendant la lecture beaucoup plus convaincante pour des décideurs humains.

Évaluation :

Justification : La vision 360°. On traite la technique, l'économie et la sécurité en un seul chat.

Indicateur

Justification

Fiabilité Factuelle	Confrontation des points de vue garantissant la vérité.
Densité de Signal	Synthèse de haut niveau (type Senior/Lead Tech).
Applicabilité (Ready-to-use)	Plan d'action complet incluant les arbitrages budgétaires.

Notre Jugement critique :

C'est la technique la plus **aboutie intellectuellement**. Elle transforme l'IA en une véritable équipe projet. Cependant, nous avons noté un écueil technique : la **dégradation du contexte**. Sur des conversations très longues, le modèle peut commencer à perdre sa posture initiale ("persona drift"). Il faut donc des transitions très sèches (ex: "FIN DE PHASE 1. DEBUT PHASE 2") pour maintenir la rigueur. C'est une méthode de **planification stratégique** de haut vol.

CHAPITRE 4 : CRITÈRES D'ÉVALUATION

Note introductive :

Évaluer l'optimisation d'un prompt nécessite de dépasser l'impression subjective de « bonne réponse ». Pour transformer cette étude expérimentale en une démarche scientifique, nous avons élaboré une grille d'évaluation multidimensionnelle. Nous confrontons ici les métriques académiques classiques aux exigences impitoyables du terrain de l'ingénierie informatique. Cette section justifie les outils de mesure utilisés pour sanctionner le succès ou l'échec de nos huit techniques expérimentées.

4.1 Les critères consensuels (état de l'art)

Dans un premier temps, nous avons repris les critères standards issus de la littérature sur le Traitement du Langage Naturel (NLP) et des présentations de référence. Ces

critères évaluent la conformité de la réponse par rapport aux attentes de base d'un utilisateur :

1. **Pertinence Thématique** : Le modèle répond-il précisément à la question posée ou s'égare-t-il dans des concepts périphériques ? (Ex: parle-t-il de la migration ou des bénéfices généraux du Cloud ?).
2. **Fiabilité et Exactitude Factuelle** : Les affirmations techniques, les numéros de versions de frameworks (Java 21) et les concepts cités sont-ils corrects ?
3. **Profondeur d'Analyse** : Le modèle traite-t-il les causes racines ou se limite-t-il aux symptômes ?
4. **Efficience (Signal/Bruit)** : Quel est le ratio entre les informations utiles et le verbiage inutile (« En tant qu'IA, je suis ravi de vous aider... »).
5. **Métriques Automatisées (BLEU/ROUGE)** : Bien que limitées, nous les mentionnons pour mesurer la proximité linguistique avec une réponse "étalon".
 - *Notre réserve critique* : Ces scores sont souvent décorrélés de la vérité technique. Un score BLEU élevé peut valider un code qui ne compile pas mais qui "ressemble" à du code.

4.2 Nos critères propres : la grille de l'expert

C'est ici que se situe l'apport majeur de notre étude. En tant qu'experts 5GI, nous avons complété la grille précédente par des critères **opérationnels** qui déterminent si le travail de l'IA a une valeur ajoutée réelle en entreprise.

1. **Déployabilité en Production (Production-Readiness)** :

C'est le critère "juge de paix". Le code généré peut-il être intégré dans un pipeline CI/CD après une simple relecture, ou nécessite-t-il une réécriture complète ? Nous évaluons ici la présence de gestion d'erreurs, de logs, de thread-safety et de respect des standards de sécurité.

2. **Rigueur vs Vraisemblance** :

Les LLM sont des champions de la vraisemblance (faire semblant de savoir). Nous notons la capacité du prompt à forcer la **vitesse de raisonnement**. Le

raisonnement est-il mathématiquement et logiquement solide, ou l'IA a-t-elle simplement "deviné" la suite probable d'un bloc de code ?

3. **Le Critère d'Audace (Innovation Destructive) :**

Le modèle a-t-il proposé une solution à laquelle l'expert n'avait pas pensé ? (Ex: un pattern de synchronisation asynchrone spécifique au lieu d'un simple appel API). Ce critère récompense les techniques (comme l'Adversarial) qui sortent l'IA du consensus "moyen".

4. **Robustesse aux Variations :**

Si nous relançons le prompt trois fois avec une température identique, la solution reste-t-elle stable ? Une technique optimisée doit réduire la variance des résultats.

4.3 La question de la notation : quantifier le jugement expert

Mettre un chiffre sur une réflexion architecturale est un exercice complexe. Pour cette étude, nous avons refusé la simplification d'un score unique sur 20. Nous avons opté pour un **système de notation hybride** :

Le Score Numérique (Quantitative) :

Chaque technique est notée sur 5 pour chacun des critères ci-dessus. Cela permet de visualiser rapidement la performance d'une technique par rapport à une autre (ex: le CROPF aura un 5/5 en structure, mais peut-être un 2/5 en audace).

Le Commentaire Qualitatif Obligatoire (Qualitative) :

Aucun score n'est valide sans une justification technique courte mais dense. Par exemple : *"Score 2/5 en sécurité car l'IA a suggéré un stockage des secrets en variables d'environnement simples, ignorant la directive de coffre-fort numérique (Vault) imposée dans le contexte."*

L'Indice de Confiance de l'Évaluateur :

Enfin, nous ajoutons une variable sur la difficulté de l'audit. Plus l'IA est "subtile" dans ses erreurs, plus l'effort d'audit humain est élevé. Une technique qui produit une réponse

complexe mais parsemée d'erreurs quasi-invisibles sera moins bien notée qu'une réponse simple et honnête.

CHAPITRE 5 : DISCUSSION ET APPORT PERSONNEL

Note introductive :

L'expérimentation de nos huit techniques sur le cas de la modernisation bancaire a permis de dégager des lois universelles de l'interaction homme-machine. Cette section s'écarte de la technique pure pour proposer une **Ingénierie de Décision**. Elle est conçue comme un guide de navigation pour l'expert qui, au-delà de la syntaxe, cherche à transformer un moteur probabiliste en un outil déterministe de précision.

5.1 L'Expert comme Variable de Contrôle : Au-delà de la Syntaxe

L'enseignement majeur de nos travaux est que l'optimisation n'est pas une question de « formulation magique », mais de **transfert de structure mentale**.

1. **Le transfert de l'expertise** : Un prompt optimisé ne crée pas d'intelligence ex-nihilo ; il agit comme un entonnoir qui canalise les connaissances du modèle. Si l'utilisateur n'injecte pas son propre vocabulaire expert, le modèle régresse vers une réponse statistique moyenne.
2. **La "Cage" Probabiliste** : L'optimisation consiste à construire une cage de contraintes autour du modèle. Chaque instruction précise réduit l'entropie de la réponse et augmente mécaniquement la probabilité d'obtenir un livrable d'une rigueur industrielle.

5.2 Guidelines Émergentes : Carte de Décision Stratégique pour tous

Selon votre objectif, vous devez choisir la technique qui équilibre le mieux l'effort de rédaction et la qualité du résultat :

- **Pour la rédaction de documents importants (Rapports, e-mails délicats, projets) :**

L'utilisation d'une structure **CROPF** garantit un résultat parfaitement organisé et professionnel, même si cela peut donner un ton un peu trop formel, un défaut que l'on corrige facilement en fournissant un exemple de votre propre style (**Few-Shot**) pour que l'IA imite votre "patte" tout en restant structurée.

- **Pour prendre une décision difficile ou tester une idée (Dilemmes, choix stratégiques) :**

Le protocole **Adversarial (Débat interne)** force l'IA à sortir de sa politesse habituelle pour pointer les risques cachés de votre projet, bien que cela demande de lui ordonner d'être volontairement critique et "méchante" pour éviter qu'elle ne se contente de valider vos envies par simple complaisance.

- **Pour effectuer des recherches sérieuses (Santé, Droit, Histoire) :**

Imposer un **Mode Hybride (Source vs Déduction)** est l'unique moyen d'éviter les inventions crédibles mais fausses de la machine, une sécurité qui rend certes la lecture plus dense mais qui s'avère payante si l'on exige que chaque conseil soit lié à une source officielle ou un fait vérifiable.

- **Pour apprendre un nouveau sujet ou trouver des idées créatives (Loisirs, curiosité) :**

Le **Prompt Naïf** (la question directe) reste le plus rapide et le plus efficace pour explorer un domaine inconnu, à condition de ne pas prendre la première réponse pour une vérité absolue et de relancer la machine dès que le résultat semble trop banal ou superficiel.

- **Pour recadrer une IA qui commence à s'égarer ou à devenir "paresseuse" :**

Dès que vous sentez que les réponses perdent en qualité au fil d'une longue discussion, le **Méta-Prompt de Reset** permet de "purger" la mémoire inutile de la conversation et de restaurer le niveau d'exigence initial en demandant au modèle de critiquer ses propres fautes avant de repartir sur des bases saines.

5.3 Synthèse : Le Pilotage de l'IA comme Compétence d'Ingénierie

Au terme de cette étude, nous affirmons que le pilotage de l'IA ne doit plus être perçu comme une simple compétence de « saisie de texte », mais comme une véritable **discipline d'ingénierie de la pensée**. L'expert humain agit désormais comme un **Architecte de Contextes**.

L'ingénieur moderne ne se contente plus de coder avec des langages informatiques classiques ; il « code » avec des intentions, des structures logiques et des garde-fous sémantiques. Cette compétence réside dans la capacité à modéliser son propre raisonnement pour l'insuffler à la machine. L'IA, bien qu'omnisciente, est une puissance de calcul aveugle : elle a besoin d'un pilote capable de décomposer une intuition complexe en instructions explicites.

À l'ENSPY, former des ingénieurs au pilotage de l'IA, c'est leur donner les clés pour devenir des « **multiplicateurs de force** ». Un ingénieur qui maîtrise l'optimisation des prompts peut diriger des légions d'agents virtuels, superviser des audits complexes en quelques minutes et garantir une sécurité factuelle là où d'autres subissent les hallucinations de la machine.

En somme, l'IA ne remplace pas l'ingénieur ; elle exige de lui une **montée en gamme intellectuelle**. Elle nous force à être plus précis, plus critiques et mieux organisés. L'excellence du résultat final n'est plus seulement dans la machine, elle est dans la finesse du lien que l'humain tisse avec elle.

7 Rapport de sources documentaires

7.1 Rapport des recherches sur internet

Le tableau suivant illustre la démarche adoptée pour notre recherche internet en faisant ressortir les temps passés, les sites visités, les requêtes saisies dans les moteurs et tous les liens visités (avec le temps moyen mis par lien).

Moteur ou source	Requêtes Soumises	Sites Visités	Liens Visités
Google	Best practices for prompt engineering OpenAI official guide	OpenAI Platform	https://platform.openai.com/docs/guides/prompt-engineering
	Few-shot prompting vs Zero-shot prompting for coding tasks	DeepLearning.AI	https://www.deeplearning.ai/short-courses/
	Prompt engineering negative constraints and jailbreaking	Medium / Towards Data Science	https://towardsdatascience.com/
	LangChain vs LlamaIndex for RAG and data indexing	StackOverflow / GitHub	https://github.com/langchain-ai/langchain
YouTube	Advanced Prompt Engineering	YouTube (Channels: AI Explained)	https://www.youtube.com/c/AIExplained
ChatGPT		Chat history	https://chatgpt.com/share/699772f1-833c-8006-9235-fc3e8bf5ff3
claude		Chat history	https://claude.ai/share/8113b88f-1b0a-4400-9167-2641cba054

8 Conclusion

Cette étude expérimentale ne se contente pas de confirmer l'impact de l'ingénierie des prompts ; elle en révèle la mécanique profonde comme un processus de transfert de structure mentale. L'optimisation n'est pas une fin en soi, mais un outil de pilotage chirurgical qui transforme un moteur probabiliste en un collaborateur déterministe. En explorant ces huit techniques sur un terrain aussi aride que la modernisation bancaire, nous avons compris que la clé réside dans la capacité de l'humain à modéliser son propre raisonnement pour l'insuffler à la machine. Le prompt devient alors un espace de contraintes négociées où chaque mot a pour fonction de réduire l'incertitude et de forcer l'émergence de l'expertise.

Savoir se servir de l'intelligence artificielle exige désormais une vision stratégique de l'instruction plutôt qu'une simple habileté rédactionnelle. Notre démarche a permis d'identifier que l'excellence opérationnelle n'est pas le fruit d'une formule unique, mais d'une orchestration de leviers variés qu'il faut savoir activer selon l'objectif visé. Pour la conception et l'architecture, il faut privilégier la dialectique et le conflit interne pour débusquer les failles invisibles ; pour l'exécution technique, il faut miser sur le mimétisme et la restriction chirurgicale pour garantir la conformité aux standards. L'art de l'optimisation consiste à savoir précisément quand brider la créativité du modèle pour obtenir de la rigueur, et quand libérer son inférence pour susciter l'innovation. C'est dans ce dosage méticuleux entre la contrainte et la liberté que se forge la véritable valeur ajoutée de l'utilisateur expert.

Au-delà de la technique, ce travail définit une nouvelle posture pour l'ingénieur de l'ENSPY : celle d'un architecte d'intentions. L'IA ne remplace pas le savoir, elle en exige une maîtrise plus profonde, car pour commander une solution de haut niveau, il faut être capable d'en définir les critères de succès avec une précision absolue. L'automatisation des tâches intellectuelles ne nous libère pas de la réflexion, elle nous impose une montée en gamme cognitive. Nous ne sommes plus de simples exécutants, mais des superviseurs de processus décisionnels complexes. La véritable puissance de cet outil réside dans son rôle de multiplicateur de force, capable de porter notre expertise vers

des échelles de production et des niveaux de fiabilité jusqu'alors inaccessibles sans une intervention humaine constante et chronophage.

En conclusion, le chemin parcouru du prompt naïf au prompt expert nous ouvre les portes d'une ingénierie symbiotique. Si l'IA est un miroir grossissant de notre propre rigueur, l'avenir appartient à ceux qui sauront construire des écosystèmes de pensée où l'humain conserve le contrôle stratégique sur la véracité et la pertinence des résultats. Cette étude marque une étape vers des systèmes de plus en plus autonomes, mais souligne avec force que la direction et l'arbitrage final resteront toujours l'apanage de celui qui maîtrise l'art de formuler les problèmes. L'enjeu de demain n'est plus seulement d'utiliser l'IA, mais de savoir l'orchestrer pour qu'elle devienne le prolongement exact de notre exigence professionnelle.

9 Bibliographie

Ressources Académiques (État de l'art du SOTA)

- **Hart, C. (1998).** *Doing a literature review: Releasing the social science research imagination.* Sage Publications. (Fondements méthodologiques de la revue de littérature).
- **Webster, J., & Watson, R. T. (2002).** *Analyzing the past to prepare for the future: Writing a literature review.* MIS Quarterly. (Structuration conceptuelle de l'état de l'art).
- **Torraco, R. J. (2005).** *Writing integrative literature reviews: Guidelines and examples.* Human Resource Development Review.
- **Kitchenham, B., & Charters, S. (2007).** *Guidelines for performing Systematic Literature Reviews in Software Engineering.* EBSE Technical Report.
- **Machi, L. A., & McEvoy, B. T. (2016).** *The literature review: Six steps to success.* Corwin.

Fondations Techniques de l'IA et du Prompting

- **Wei, J., et al. (2022).** *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* arXiv preprint arXiv:2201.11903. (Référence majeure sur la technique CoT).
- **Bender, E. M., et al. (2021).** *On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?* Proceedings of the ACM Conference on Fairness, Accountability, and Transparency.
- **Bommasani, R., et al. (2021).** *On the Opportunities and Risks of Foundation Models.* Stanford Center for Research on Foundation Models.
- **Kasneci, E., et al. (2023).** *ChatGPT for good? On opportunities and challenges of large language models for education.* Learning and Individual Differences.
- **OpenAI. (2024).** *Prompt Engineering Guide.* Consulté sur : <https://platform.openai.com/docs/guides/prompt-engineering>

Références du Cas d'Étude (IT & Architecture)

- **Evans, E. (2003).** *Domain-Driven Design: Tackling Complexity in the Heart of Software.* Addison-Wesley Professional.
- **Richardson, C. (2018).** *Microservices Patterns: With examples in Java.* Manning Publications.
- **Newman, S. (2019).** *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith.* O'Reilly Media.

- **Oracle. (2023).** *Java 21 Language Documentation & New Features*. Consulté sur : <https://docs.oracle.com/en/java/javase/21>

10 Glossaire

- **Adversarial Prompting (Prompting Adversarial)** : Technique consistant à utiliser l'IA contre elle-même ou à la soumettre à des scénarios de test critiques pour identifier ses failles de raisonnement, ses biais ou ses hallucinations.
- **AIPRM (Artificial Intelligence Prompt Engineering Resource Manager)** : Extension de navigateur fournissant une bibliothèque communautaire de templates de prompts optimisés pour divers métiers (SEO, Marketing, Code).
- **Bounded Context (Contexte Borné)** : Concept central du Domain Driven Design (DDD) définissant une limite explicite à l'intérieur de laquelle un modèle métier est applicable. Essentiel pour le découpage de monolithes.
- **Chain-of-Thought (CoT - Chaîne de Pensée)** : Technique de prompting incitant le modèle à décomposer son raisonnement en étapes logiques intermédiaires avant de formuler une réponse finale, réduisant ainsi les erreurs de calcul ou de logique.
- **CROPF** : Framework de structuration de prompt regroupant les composantes : Contexte, Rôle, Objectif, Procédure et Format.
- **CVE (Common Vulnerabilities and Exposures)** : Système de référencement mondial des failles de sécurité informatique connues.
- **DDD (Domain-Driven Design)** : Approche de conception logicielle centrée sur le métier, utilisée dans notre étude pour guider l'IA dans le découpage cohérent des microservices.
- **Dérive de Contexte (Context Drift)** : Tendance d'un modèle à perdre le fil des instructions initiales au fur et à mesure d'une conversation longue.
- **Few-Shot Prompting** : Méthode consistant à fournir un ou plusieurs exemples de paires "Entrée/Sortie" à l'IA pour lui apprendre un format ou un niveau d'exigence spécifique sans réentraînement.
- **Factualité (Grounding)** : Capacité d'un prompt à "ancrer" les réponses du modèle dans des sources de données réelles et vérifiables, limitant l'imagination statistique.
- **Hallucination** : Phénomène par lequel un LLM génère des informations factuellement fausses mais linguistiquement plausibles.
- **Inférence** : Processus par lequel le modèle déduit une conclusion ou une solution à partir des informations fournies dans le prompt et de ses connaissances pré-entraînées.
- **ISO 20022** : Norme internationale pour l'échange de messages financiers, utilisée comme critère de vérité factuelle dans nos tests bancaires.

- **Legacy Code (Code Hérité)** : Code source ancien (dans notre étude Java 8) souvent dépourvu de tests automatisés et difficile à maintenir, constituant le terrain de notre migration.
- **LLM (Large Language Model)** : Modèle de langage entraîné sur de vastes volumes de données (ex: GPT-4, Claude 3) capable de traiter et de générer du texte complexe.
- **Méta-Prompt** : Prompt conçu pour agir sur un autre prompt ou sur une réponse précédente (ex: un prompt d'audit ou de réécriture).
- **Optimisation Sémantique** : Ajustement de la syntaxe et du vocabulaire d'une instruction pour aligner l'espace latent du modèle sur l'expertise attendue.
- **Outbox Pattern** : Pattern de conception utilisé en microservices pour garantir la cohérence des données entre une base de données et un système de messagerie (ex: Kafka), identifié par nos prompts experts.
- **Prompt Naïf (Baseline)** : Instruction simple, directe et sans structure, utilisée comme point de comparaison pour mesurer l'efficacité de l'optimisation.
- **RAG (Retrieval-Augmented Generation)** : Technique consistant à fournir des documents externes au modèle au moment de l'inférence pour augmenter sa base de connaissances en temps réel.
- **SAGA Pattern** : Pattern architectural de gestion des transactions distribuées à travers plusieurs microservices.
- **Strangler Fig Pattern (Pattern de l'Étrangleur)** : Stratégie de migration logicielle consistant à remplacer progressivement les fonctionnalités d'un monolithe par de nouveaux services.
- **Sycophantie (Sycophancy)** : Biais des modèles de langage consistant à adapter leurs réponses pour plaire à l'utilisateur ou confirmer ses erreurs, au détriment de la vérité technique.
- **Token** : Unité de base de traitement d'un LLM (environ 0,75 mot). La longueur des prompts est limitée par une "fenêtre de tokens".
- **Zero-Shot Prompting** : Demander au modèle d'exécuter une tâche sans lui fournir aucun exemple préalable.